

Final Project Proposal

Prediction to Prescription: Building an AI Agent

Team: 1 **Project Title:** Predicting Marketing Conversions **Date:** 31 May 2026

Name	Email	Program	Primary Role
Member 1 Shubh Vaishnav	vaishnav@purdue.edu	MSBAIM	Project Owner
Member 2 Nick Sopcic	nsopcic@purdue.edu	MSBAIM	Component owner
Member 3 Ethin DalPian	edalpian@purdue.edu	MSBAIM	Data Collection & Preparation
Member 4 Matt Foor	foor0@purdue.edu	MSBAIM	Component owner

Section 1 — Problem Statement and Business Context

Describe the class of business problem your agent will tackle — in plain language, without writing down a formal optimization model. Your agent will generate the problem formulation; your job here is to give it a clear, well-motivated problem to work on. Aim for 1–1.5 pages across the three subsections below.

1.1 Business Context

Our agent is designed for a mid-sized e-commerce retailer that sells consumer goods online. The marketing manager decides how to allocate a fixed weekly advertising budget across four digital channels: search ads (Google/Bing), social media (Facebook/Instagram), display (programmatic banners), and affiliate marketing. Currently, the manager uses a simple rule-of-thumb: *40% to search, 30% to social, 20% to display, and 10% to affiliate*. This static allocation ignores channel-specific diminishing returns, cross-channel interactions, and weekly changes in performance. As a result, the firm often overspends on saturated channels and misses opportunities to reallocate budget to channels with higher marginal lift. The marketing team re-evaluates the budget every month, but manual analysis is slow and prone to cognitive biases (e.g., recency bias toward last week's high-performing channel).

1.2 Problem Class

This is a *budget allocation problem with diminishing returns*: Each week, the manager chooses a spend amount for each channel (continuous, non-negative). The goal is to maximize total conversions (or revenue) generated across channels. This decision is limited by a fixed total budget and optional per-channel maximum spend limits (e.g., to avoid over-exposure). The uncertainty lies in the response curves — the relationship between spend and conversions is unknown and must be estimated from historical data. This is a concave optimization problem because each channel exhibits saturation (marginal gains decrease as spend increases).

1.3 Why This Problem Is Hard Without an Agent

The problem is hard without an agent for three reasons. Firstly, demand uncertainty: the conversion lift per dollar on each channel varies over time due to seasonality, competition, and changing user behavior. A human manager cannot reliably estimate marginal response rates from noisy weekly data. Second, interacting constraints: the budget must be split among multiple channels, and the optimal allocation depends on the relative effectiveness of each channel; It's a textbook resource allocation problem that becomes nonlinear when diminishing returns are present. Third, frequency of change: optimal allocations can shift week-to-week; manually re-optimizing is impractical. The agent will outperform current practice by automatically estimating channel-specific response curves from data, solving the constrained optimization exactly, and adapting recommendations as new data arrives. Success will be measured by a higher total conversion count on hold-out test periods compared to the static rule.

Section 2 — Data Plan

Identify every real dataset you will use. For each one, provide the source, approximate size, key features, and how it connects to the uncertain parameters in your model. Explain your train/test split strategy. Aim for 0.5–1 page.

2.1 Primary Dataset

Field	Details
Dataset name	Multi-Touch Attribution
Source / URL	https://www.kaggle.com/datasets/vivekparasharr/multi-touch-attribution/data
Approximate size	10K Rows $\times$ 5 columns, Collected over 2 days
Key features	Unique Users: 2,847, Most Frequent Channel Direct Traffic (17.2%)
Outcome variable	Attribution modeling/Conversion modeling
Known limitations	Campaign Involvement: 31.3% of interactions had no campaign assigned. mix).

2.2 Additional Datasets (if any)

<https://huggingface.co/datasets/criteo/criteo-attribution-dataset>
(This is a backup dataset)

2.3 Train / Test Split

Approximately 85% of the data will be used for training and 15% for testing. Our team plans to hold out 15% (1,500 rows) as a final test set then run k-fold cross-validation on the remaining 8,500 for all our model selection and hyperparameter tuning. This uses our data far more efficiently than

a single fixed validation set, because every training row gets to serve as validation across the folds, and our performance estimate during tuning is averaged rather than dependent on one arbitrary 1,500-row slice.

Section 3 — Methods Sketch

Outline the technical approach for each agent component. You do not need final answers — but you must show end-to-end thinking from data input to decision output. Address prediction, optimization, constraint handling, and baseline. Aim for 1–1.5 pages.

3.1 Prediction Layer

The optimizer in Section 3.2 needs a smooth, concave response curve for each channel, while the raw data is a binary conversion flag recorded at the individual touchpoint level. To bridge that gap we use a two stage hybrid design rather than a single model. The first stage learns where conversions come from at fine granularity, and the second stage compresses that signal into the concave channel curves the optimizer consumes.

Stage 1: Touchpoint propensity model. We train a gradient boosted classifier (XGBoost) to estimate the probability that a touchpoint converts, $P(\text{convert} \mid \mathbf{x})$, where the feature vector $\mathbf{x}$ covers channel, campaign, recency, and journey position. XGBoost is chosen here because it captures the channel by campaign and timing interactions that a linear model would miss, handles the categorical and skewed features in this dataset with minimal manual engineering, and produces well calibrated scores after isotonic or Platt calibration. A regularized logistic regression is fit alongside it as an interpretable baseline and a sanity check on the boosted scores. The objective for this stage is standard log loss (binary cross entropy), the natural loss for calibrated probability estimates.

Stage 2: Channel response curves. We aggregate the Stage 1 predictions to the channel level, pairing exposure volume per channel with expected conversions, then fit a concave saturation curve for each channel c . We use a Hill saturation form, $g_c(x) = \frac{V_c x^{n_c}}{K_c^{n_c} + x^{n_c}}$, with a logarithmic curve as a fallback, fit per channel by nonlinear least squares using `scipy.optimize.curve_fit`. This stage deliberately enforces the structure the optimizer requires: the fitted curves are monotone and concave by construction, smooth, and extrapolate sensibly beyond the observed volume range, none of which a raw tree ensemble guarantees. The fitted parameters V_c (saturation ceiling) and K_c (half saturation point) are the uncertain parameters passed to the optimizer, and budget enters through an assumed cost per exposure (CPM or CPC) that maps dollars to volume x .

A single XGBoost model on its own is a poor fit for the prescription step. A tree ensemble outputs a step function that is non concave and non monotone by default, so feeding it to a convex solver is invalid and feeding it to a general solver yields unstable allocations that sit on flat or jagged regions of the surface. Imposing monotone constraints plus post hoc smoothing on XGBoost would recover roughly what the parametric curve already gives us, but with less interpretability and no extrapolation. We therefore let XGBoost do what it is strong at, prediction, and let the parametric layer supply a clean concave object for optimization.

On the choice of loss, a fully end to end decision aware objective (for example an SPO style loss

that propagates allocation regret back through the prediction) is a reasonable extension, but we keep standard losses for the proposal. Log loss in Stage 1 and nonlinear least squares in Stage 2 are sufficient because the concave Stage 2 curve already aligns the prediction target with the downstream concave objective, the two stage seam keeps the pipeline transparent and easy to validate, and the modular structure is far easier to reparameterize for the Section 5.2 stakeholder change. Decision aware training is flagged as a stretch goal if time permits.

3.2 Optimization Engine

We solve the weekly allocation with `scipy.optimize.minimize` using Sequential Least Squares Programming (SLSQP), which accepts both the budget equality constraint and the per channel bound constraints directly. The objective is to maximize total expected conversions, $\sum_c g_c(x_c)$, where each g_c is the concave Hill curve fit in Section 3.1; because the solver minimizes, we pass it the negative of this sum. The summed objective is concave and the feasible region is convex, so the problem has a single global optimum in principle. In practice we wrap the solver in a multistart loop: the optimizer runs from several random feasible starting allocations and the best objective is kept. Multistart costs little here and protects against two real risks, numerical flat spots where a fitted curve has saturated and any mild non concavity introduced when a curve is fit imperfectly or replaced by its logarithmic fallback. The result is a robust gradient based search that does not assume the curves are perfectly well behaved. We keep CVXPY in reserve as a convex check: when the fitted curves are cleanly concave, CVXPY should reproduce the SLSQP optimum, which is a useful validation of both layers.

3.3 Constraint Handling

We enforce every constraint as a hard constraint inside the solver rather than through penalty terms, so each allocation the agent proposes is feasible by construction. There are three. First, a budget constraint, total spend across the four channels equals the weekly budget, $\sum_c x_c = B$, passed to SLSQP as an equality constraint. Second, per channel bounds, each channel has a lower bound of zero and an optional upper cap, $0 \leq x_c \leq u_c$, passed as box bounds to prevent over exposure on any single channel. Third, non negativity, which the lower bound already guarantees. We prefer hard constraints over penalty terms because penalties require tuning a weight, can leave small residual violations, and would muddy the explanation the agent gives the stakeholder. If a future variant introduces a constraint SLSQP cannot express directly, for example an integer or logical condition, we would move that piece to a projection step or switch to a solver that supports it.

3.4 Baseline Comparison

The current practice we aim to beat is the firm’s static rule from Section 1.1: a fixed split of 40% search, 30% social, 20% display, and 10% affiliate, applied every week regardless of recent performance. We also include a second, slightly stronger baseline, a greedy heuristic that pushes the full budget toward whichever channel converted best in the previous week, since that is the kind of recency driven move a manager actually makes. Both baselines are evaluated on the same hold out weeks as the agent and scored with the same fitted response curves, so the only thing that differs is the allocation rule. Improvement is measured as the increase in total expected conversions of the agent’s allocation over each baseline on the hold out set. Reporting against both a naive fixed rule

and a greedy heuristic shows the gain is not merely beating an obviously poor policy.

3.5 Pipeline Diagram

The data to decision flow has five stages. Touchpoint records feed the Stage 1 classifier, whose per channel conversion estimates are aggregated and fit into concave response curves, which become the parameters the optimizer searches over, producing a weekly budget allocation together with a plain language explanation for the stakeholder.

Touchpoint data → **XGBoost propensity (Stage 1)** → **Hill response curves (Stage 2)** → **scipy multistart optimizer** → **Allocation + explanation**

Section 4 — Team Roles

Assign each member a primary component. Every one of the five components must be covered. Roles are not binding, but each member must have clear ownership.

Component	Primary Owner	Brief Description of Responsibility
Problem Formulation Module	Shubh Vaishnav	Translates the business problem into decision variables, an objective, and constraints, and maintains the structured problem schema the agent consumes.
Data Collection & Preparation	Ethin DalPian	Sources and cleans the attribution data, engineers channel, campaign, and recency features, and builds the per channel exposure aggregates.
Prediction Layer	Nick Sopcic	Builds the two stage predictor, an XGBoost touchpoint propensity model and the concave Hill response curves passed to the optimizer.
Optimization Engine	Matt Foor	Implements the scipy multistart solver, encodes the budget and per channel constraints, and returns the optimal weekly allocation.
Explanation & Validation	Shubh Vaishnav	Runs the bootstrap uplift test against the baselines and produces the plain language rationale the agent reports to the stakeholder.

Of course our team will collaborate with each other throughout the project so shared ownership is always implied.

Section 5 — Benchmark and Validation Plan

Explain how you will evaluate success on (a) your own problem and (b) the instructor benchmark. For (a), state the metric and how you will interpret it. For (b), explain how your agent can accept a new problem description as input. Aim for 0.5 pages.

5.1 Evaluating Your Own Problem

Our primary metric is total expected conversions on the held out weeks. We compare the agent’s allocation against the two baselines from Section 3.4 on the same weeks and the same fitted curves, so any difference reflects the allocation rule alone. Because a single point estimate of uplift can be noisy on a small hold out, we establish success through a bootstrap: we resample the hold out weeks with replacement many times, recompute the conversion uplift of the agent over each baseline on every resample, and build a confidence interval for that uplift. We treat the result as a meaningful win when the lower bound of the interval stays above zero, meaning the agent beats the baseline consistently rather than on a lucky split, rather than fixing an arbitrary percentage threshold in advance. We report the median uplift and the interval rather than a single number, and we also report the realized allocation so the size and direction of the budget shift are visible to the stakeholder.

5.2 Stakeholder Robustness

Our agent is built so that the business reality lives in data, not in code. The pipeline reads a structured problem specification, and the model and solver are written against that specification rather than against fixed numbers.

- **What is parameterized.** The problem specification holds the weekly budget, the per channel cost assumptions (CPM or CPC), the per channel upper caps, the objective sense and target (for example maximize total conversions, or minimize the spend needed to reach a conversion target), and the set of decision channels. The response curves, the constraint set, and the objective passed to the solver are all built from these fields at run time. Only the structural choices, the two stage architecture and the concave curve family, are fixed in code.
- **How a new problem reaches the agent.** We use two layers. A human can edit the specification directly as a configuration dictionary, which is fast and unambiguous. In parallel, an LLM reads the instructor’s natural language problem change and emits the same specification, so a plain English instruction such as a new cap on display spend or a shift to a conversions per dollar objective is translated into the structured fields automatically. A validation step checks the LLM output against the schema, the allowed ranges, and feasibility before anything reaches the optimizer, so a malformed or contradictory change is caught rather than silently solved.
- **What the limits are.** Changes expressed in the existing vocabulary are handled gracefully: a changed budget, a new or tightened per channel cap, an updated cost assumption, or a switch between supported objectives all require no code change and solve again in seconds. Changes outside that vocabulary need real work. A brand new decision dimension (for example choosing creative as well as spend), a non concave or combinatorial objective that breaks the concavity

our solver relies on, or a constraint that needs data we do not have would require rebuilding the prediction or optimization layer, not just a new specification. We will be explicit with the stakeholder about which bucket a requested change falls into and what the business cost of the resulting reallocation is.

References (if any)

- [1] Criteo AI Lab. (2016). Criteo Attribution Dataset for Bidding. <https://ailab.criteo.com/criteo-attribution-modeling-bidding-dataset/>
- [2] Parashar, V. Multi-Touch Attribution Dataset. Kaggle. <https://www.kaggle.com/datasets/vivekparasharr/multi-touch-attribution>
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [4] Jin, Y., Wang, Y., Sun, Y., Chan, D., & Koehler, J. (2017). *Bayesian Methods for Media Mix Modeling with Carryover and Shape Effects*. Google Inc.